MODUL PRAKTIKUM

PEMROGRAMAN MOBILE

Pertemuan 2

StyleSheet, Flexbox Layout & Event Handling

Menguasai layout mobile dan interaksi pengguna di React Native

Mata Kuliah **Pemrograman Mobile**	Pertemuan / Topik **2 — StyleSheet, Flexbox & Events**
Prasyarat **Pertemuan 1 — JSX, Components, Props, State**	Estimasi Waktu **100 menit (2 sks praktikum)**

Identitas Mahasiswa

Nama Lengkap	:	
NIM	:	
Kelas / Kelompok	:	
Tanggal Praktikum	:	
Nama Asisten / Dosen	:	
Nilai	:	

Tujuan Praktikum

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Menggunakan StyleSheet.create() untuk mendefinisikan style komponen React Native
2. Menerapkan konsep Flexbox untuk menyusun layout komponen secara vertikal dan horizontal
3. Menggunakan properti Flexbox: flexDirection, justifyContent, alignItems, flex, flexWrap
4. Membedakan dan menerapkan berbagai jenis event handler: onPress, onLongPress, onChange, onFocus, onBlur
5. Menggunakan komponen Pressable sebagai pengganti TouchableOpacity yang lebih fleksibel
6. Menangani gesture dasar: tap, long press, dan input keyboard
7. Membangun layout halaman mobile yang rapi dan responsif
8. Menggabungkan layout, styling, dan event handling dalam satu aplikasi mobile yang utuh

Alat dan Bahan

Pastikan semua hal berikut sudah siap sebelum memulai:

- Project Expo dari Pertemuan 1 (atau buat project baru: npx create-expo-app praktikum-p2 --template blank)
- Emulator Android Studio AVD sudah menyala
- VS Code dengan ekstensi React Native Tools
- Expo Dev Server berjalan (npx expo start, tekan 'a')

Materi Praktikum

BAGIAN 01

StyleSheet & Styling

Cara mendefinisikan dan menerapkan style di React Native

Bagian 1: StyleSheet

1.1 Cara Kerja StyleSheet di React Native

Di React Native tidak ada CSS file. Semua style didefinisikan menggunakan objek JavaScript melalui StyleSheet.create(). Sintaksnya mirip CSS namun menggunakan camelCase dan nilai numerik (tanpa satuan px).

CSS (Web)	StyleSheet React Native	Keterangan
`background-color: blue`	backgroundColor: 'blue'	camelCase, nilai string
`font-size: 16px`	fontSize: 16	Angka tanpa satuan 'px'
`border-radius: 8px`	borderRadius: 8	Angka tanpa satuan
`margin-top: 10px`	marginTop: 10	camelCase, tanpa 'px'
`padding: 16px`	padding: 16	Shorthand tetap bisa dipakai
`flex-direction: row`	flexDirection: 'row'	camelCase, nilai string
`text-align: center`	textAlign: 'center'	Hanya berlaku di <Text>
`display: flex`	(default semua View)	Semua View sudah flex

1.2 Contoh StyleSheet Lengkap

```
import { StyleSheet, View, Text } from 'react-native';

export default function KartuInfo() {
  return (
    <View style={styles.kartu}>
      <View style={styles.header}>
        <Text style={styles.judul}>Informasi Mahasiswa</Text>
      </View>
      <View style={styles.isi}>
        <Text style={styles.label}>Nama</Text>
        <Text style={styles.nilai}>Budi Santoso</Text>
      </View>
      <View style={styles.isi}>
        <Text style={styles.label}>NIM</Text>
        <Text style={styles.nilai}>20230001</Text>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  kartu: {
    backgroundColor: '#FFFFFF',
    borderRadius: 12,
    borderWidth: 1,
    borderColor: '#BDD7EE',
    marginHorizontal: 16,
    marginVertical: 8,
    overflow: 'hidden',
    // Shadow (iOS)
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.1,
    shadowRadius: 4,
    // Shadow (Android)
    elevation: 3,
  },
  header: {
    backgroundColor: '#1F4E79',
```

```
    paddingVertical: 10,
    paddingHorizontal: 16,
  },
  judul: { color: '#FFFFFF', fontSize: 16, fontWeight: 'bold' },
  isi: {
    flexDirection: 'row',
    paddingHorizontal: 16,
    paddingVertical: 8,
    borderBottomWidth: 1,
    borderBottomColor: '#F0F0F0',
  },
  label: { flex: 1, fontSize: 14, color: '#888888' },
  nilai: { flex: 2, fontSize: 14, color: '#1F4E79', fontWeight: '500' },
});
```

Tips: Inline Style vs StyleSheet

Inline style: <View style={{ backgroundColor: 'red', padding: 8 }}>
-> Mudah ditulis tapi lebih lambat (objek baru dibuat setiap render)

StyleSheet.create(): const styles = StyleSheet.create({ box: { ... } })
-> Direkomendasikan: divalidasi saat startup, lebih performant

Style array (gabungkan dua style):
<View style={[styles.dasar, styles.tambahan, { margin: 8 }]}>
-> Urutan kanan menimpa kiri jika ada properti yang sama

BAGIAN 02

Flexbox Layout

Sistem layout utama React Native untuk menyusun komponen

Bagian 2: Flexbox Layout

2.1 Konsep Dasar Flexbox React Native

React Native menggunakan Flexbox sebagai satu-satunya sistem layout. Semua komponen View secara default adalah flex container. Perbedaan penting dengan CSS web: default flexDirection di React Native adalah 'column' (vertikal), bukan 'row'.

Properti	Nilai Umum	Fungsi
`flexDirection`	'column' | 'row'	Arah susun anak: vertikal (default) atau horizontal
`justifyContent`	'flex-start' | 'center' | 'flex-end' | 'space-between' | 'space-around' | 'space-evenly'	Distribusi anak di sumbu UTAMA
`alignItems`	'flex-start' | 'center' | 'flex-end' | 'stretch'	Posisi anak di sumbu SILANG
`flex`	angka (1, 2, dst.)	Proporsi ruang yang diambil komponen
`flexWrap`	'wrap' | 'nowrap'	Izinkan anak pindah baris jika penuh
`alignSelf`	'auto' | 'flex-start' | 'center' | 'flex-end'	Override alignItems untuk satu anak
`gap`	angka	Jarak antar anak (React Native 0.71+)

2.2 Pola Layout yang Sering Digunakan

```
// --- POLA 1: Tengah secara horizontal dan vertikal ---
container: {
  flex: 1,
  justifyContent: 'center',   // tengah di sumbu vertikal
  alignItems: 'center',       // tengah di sumbu horizontal
},

// --- POLA 2: Header - Konten - Footer ---
// Parent flex: 1
header:  { height: 60, backgroundColor: '#1F4E79' },
konten:  { flex: 1 },           // mengisi sisa ruang
footer:  { height: 50, backgroundColor: '#F0F0F0' },

// --- POLA 3: Item sejajar horizontal dengan space-between ---
baris: {
  flexDirection: 'row',
  justifyContent: 'space-between',
  alignItems: 'center',
  paddingHorizontal: 16,
},

// --- POLA 4: Grid 2 kolom (menggunakan flexWrap) ---
grid: {
```

```
    flexDirection: 'row',
    flexWrap: 'wrap',
    gap: 8,
  },
  gridItem: {
    width: '48%',              // ~2 kolom dengan gap
  },

  // --- POLA 5: Ikon kiri + teks kanan ---
  itemList: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 12,
    paddingVertical: 10,
  },
```

2.3 Contoh Implementasi Layout Halaman

```
import { StyleSheet, View, Text, ScrollView } from 'react-native';

export default function HalamanUtama() {
  return (
    // flex: 1 agar mengisi seluruh layar
    <View style={styles.layar}>

      {/* HEADER — tinggi tetap */}
      <View style={styles.header}>
        <Text style={styles.headerJudul}>Aplikasi Mahasiswa</Text>
        <Text style={styles.headerSub}>Selamat Datang</Text>
      </View>

      {/* KONTEN — mengisi sisa ruang, bisa di-scroll */}
      <ScrollView style={styles.konten}>
        <View style={styles.grid}>
          <View style={[styles.kartu, { backgroundColor: '#DAEEF3' }]}>
            <Text style={styles.kartuAngka}>24</Text>
            <Text style={styles.kartuLabel}>Mata Kuliah</Text>
          </View>
          <View style={[styles.kartu, { backgroundColor: '#E2EFDA' }]}>
            <Text style={styles.kartuAngka}>3.85</Text>
            <Text style={styles.kartuLabel}>IPK</Text>
          </View>
          <View style={[styles.kartu, { backgroundColor: '#FFF2CC' }]}>
            <Text style={styles.kartuAngka}>6</Text>
            <Text style={styles.kartuLabel}>Semester</Text>
          </View>
          <View style={[styles.kartu, { backgroundColor: '#FCE4D6' }]}>
            <Text style={styles.kartuAngka}>12</Text>
            <Text style={styles.kartuLabel}>SKS Tersisa</Text>
          </View>
        </View>
      </ScrollView>

      {/* FOOTER — tinggi tetap */}
      <View style={styles.footer}>
        <Text style={styles.footerTeks}>Universitas — 2024</Text>
      </View>

    </View>
  );
}

const styles = StyleSheet.create({
  layar:        { flex: 1, backgroundColor: '#F5F5F5' },
```

```
  header:        { backgroundColor: '#1F4E79', paddingTop: 50, paddingBottom:
20, paddingHorizontal: 20 },
  headerJudul:   { color: '#FFFFFF', fontSize: 22, fontWeight: 'bold' },
  headerSub:     { color: '#BDD7EE', fontSize: 14, marginTop: 4 },
  konten:        { flex: 1, padding: 16 },
  grid:          { flexDirection: 'row', flexWrap: 'wrap', gap: 12 },
  kartu:         { width: '47%', borderRadius: 12, padding: 16, alignItems:
'center' },
  kartuAngka:    { fontSize: 32, fontWeight: 'bold', color: '#1F4E79' },
  kartuLabel:    { fontSize: 12, color: '#666666', marginTop: 4 },
  footer:        { backgroundColor: '#EEEEEE', padding: 12, alignItems: 'center'
},
  footerTeks:    { color: '#888888', fontSize: 12 },
});
```

BAGIAN 03

Event Handling

Menangani interaksi pengguna: tap, press, input, dan gesture

Bagian 3: Event Handling

3.1 Jenis-Jenis Event di React Native

Event	Komponen	Keterangan
`onPress`	TouchableOpacity, Pressable, Button	Dieksekusi saat tombol ditekan (tap)
`onLongPress`	TouchableOpacity, Pressable	Dieksekusi saat tombol ditekan lama (>500ms)
`onPressIn`	Pressable	Dieksekusi saat jari mulai menyentuh
`onPressOut`	Pressable	Dieksekusi saat jari diangkat
`onChange`	TextInput	Dipanggil setiap karakter berubah (membawa event object)
`onChangeText`	TextInput	Dipanggil setiap karakter berubah (langsung nilai string)
`onFocus`	TextInput	Input mendapat fokus (keyboard muncul)
`onBlur`	TextInput	Input kehilangan fokus (keyboard sembunyi)
`onSubmitEditing`	TextInput	Tombol Enter/Done di keyboard ditekan
`onScroll`	ScrollView, FlatList	Dipanggil saat konten di-scroll

3.2 TouchableOpacity vs Pressable

React Native menyediakan dua komponen utama untuk menangani sentuhan. Pressable adalah komponen yang lebih baru dan lebih fleksibel.

```
import { TouchableOpacity, Pressable, Text, Alert } from 'react-native';

// --- TouchableOpacity (cara lama, masih banyak dipakai) ---
// Otomatis memberikan efek opacity saat ditekan
<TouchableOpacity
  onPress={() => Alert.alert('Info', 'Tombol ditekan!')}
  onLongPress={() => Alert.alert('Info', 'Tombol ditekan lama!')}
  activeOpacity={0.7}     // opacity saat ditekan (0-1)
  style={styles.tombol}
>
  <Text style={styles.teks}>Tekan Saya</Text>
</TouchableOpacity>

// --- Pressable (cara baru, lebih fleksibel) ---
// Style bisa berubah berdasarkan state tekan
<Pressable
  onPress={() => Alert.alert('Info', 'Ditekan!')}
  onLongPress={() => Alert.alert('Info', 'Ditekan lama!')}
  onPressIn={() => console.log('Mulai ditekan')}
```

```
    onPressOut={() => console.log('Dilepas')}
    style={({ pressed }) => [
      styles.tombol,
      { backgroundColor: pressed ? '#1F4E79' : '#2E75B6' },
    ]}
  >
    {({ pressed }) => (
      <Text style={styles.teks}>
        {pressed ? 'Sedang Ditekan...' : 'Tekan Saya'}
      </Text>
    )}
  </Pressable>
```

Kapan Pakai Mana?

TouchableOpacity -> Cukup untuk sebagian besar tombol sederhana

Pressable -> Gunakan jika butuh kontrol penuh atas tampilan saat ditekan, atau butuh onPressIn / onPressOut

Button -> Komponen bawaan, tampilannya mengikuti OS — kurang fleksibel untuk styling custom

3.3 Event Handling pada TextInput

```
import { useState } from 'react';
import { View, Text, TextInput, StyleSheet } from 'react-native';

export default function FormInput() {
  const [nama, setNama]          = useState('');
  const [email, setEmail]        = useState('');
  const [isFokus, setIsFokus]    = useState(false);

  return (
    <View style={styles.form}>

      {/* onChangeText: langsung terima string baru */}
      <Text style={styles.label}>Nama Lengkap</Text>
      <TextInput
        style={styles.input}
        value={nama}
        onChangeText={(teks) => setNama(teks)}
        placeholder='Masukkan nama lengkap'
        placeholderTextColor='#AAAAAA'
      />

      {/* onFocus & onBlur: mengubah tampilan saat aktif */}
      <Text style={styles.label}>Email</Text>
      <TextInput
        style={[styles.input, isFokus && styles.inputFokus]}
        value={email}
        onChangeText={setEmail}
        onFocus={() => setIsFokus(true)}
        onBlur={() => setIsFokus(false)}
        placeholder='nama@email.com'
        placeholderTextColor='#AAAAAA'
        keyboardType='email-address'
        autoCapitalize='none'
      />

      <Text style={styles.preview}>
        Preview: {nama} — {email}
      </Text>
    </View>
```

```
  );
}

const styles = StyleSheet.create({
  form:       { padding: 20 },
  label:      { fontSize: 14, fontWeight: '600', color: '#333', marginBottom: 6
},
  input: {
    borderWidth: 1, borderColor: '#CCCCCC', borderRadius: 8,
    paddingHorizontal: 14, paddingVertical: 10,
    fontSize: 15, backgroundColor: '#FFFFFF', marginBottom: 16,
  },
  inputFokus: { borderColor: '#2E75B6', borderWidth: 2 },
  preview:    { marginTop: 12, color: '#666666', fontStyle: 'italic' },
});
```

Prop TextInput	Nilai	Fungsi
`keyboardType`	'default' | 'numeric' | 'email-address' | 'phone-pad'	Jenis keyboard yang muncul
`secureTextEntry`	true | false	Sembunyikan teks (untuk password)
`multiline`	true | false	Input bisa multi-baris
`maxLength`	angka	Batas maksimal karakter
`autoCapitalize`	'none' | 'sentences' | 'words' | 'characters'	Aturan huruf kapital otomatis
`returnKeyType`	'done' | 'next' | 'search' | 'go'	Label tombol Enter di keyboard

Langkah-Langkah Praktikum

Praktikum A — Eksplorasi Flexbox Layout

Target & Waktu

Memahami cara kerja Flexbox secara visual dengan membuat layout eksperimen.

Perkiraan waktu: 25 menit

1 Siapkan project

Buka project dari pertemuan 1, atau buat project baru. Pastikan emulator sudah menyala dan Expo Dev Server berjalan.

```
# Jika buat project baru:
npx create-expo-app praktikum-p2 --template blank
cd praktikum-p2
npx expo start
# Tekan 'a' untuk buka di Android emulator
```

2 Buat file FlexboxDemo.js

Buat folder components/ lalu buat file FlexboxDemo.js. Tulis kode berikut untuk mengeksplorasi berbagai konfigurasi Flexbox:

```
import { View, Text, StyleSheet, ScrollView } from 'react-native';

// Komponen kotak kecil untuk eksperimen
function Kotak({ warna, label }) {
  return (
    <View style={[styles.kotak, { backgroundColor: warna }]}>
      <Text style={styles.kotakTeks}>{label}</Text>
    </View>
  );
}

// Komponen container eksperimen
function Demo({ judul, containerStyle }) {
  return (
    <View style={styles.demoWrapper}>
      <Text style={styles.demoJudul}>{judul}</Text>
      <View style={[styles.demoKonten, containerStyle]}>
        <Kotak warna='#2E75B6' label='A' />
        <Kotak warna='#1F4E79' label='B' />
        <Kotak warna='#BDD7EE' label='C' />
      </View>
    </View>
  );
}

export default function FlexboxDemo() {
  return (
    <ScrollView style={styles.layar}>
      <Text style={styles.pageJudul}>Eksplorasi Flexbox</Text>

      <Demo judul='1. column (default)' containerStyle={{ flexDirection:
'column' }} />
      <Demo judul='2. row' containerStyle={{ flexDirection: 'row' }} />
      <Demo judul='3. row + justifyContent: center' containerStyle={{
flexDirection: 'row', justifyContent: 'center' }} />
      <Demo judul='4. row + justifyContent: space-between' containerStyle={{
flexDirection: 'row', justifyContent: 'space-between' }} />
      <Demo judul='5. row + justifyContent: space-around' containerStyle={{
flexDirection: 'row', justifyContent: 'space-around' }} />
```

```
      <Demo judul='6. row + alignItems: flex-end' containerStyle={{
flexDirection: 'row', alignItems: 'flex-end', height: 100 }} />
      <Demo judul='7. row + alignItems: center' containerStyle={{
flexDirection: 'row', alignItems: 'center', height: 100 }} />
    </ScrollView>
  );
}

const styles = StyleSheet.create({
  layar:       { flex: 1, backgroundColor: '#F5F5F5', paddingTop: 50 },
  pageJudul:   { fontSize: 22, fontWeight: 'bold', color: '#1F4E79', textAlign:
'center', marginBottom: 16 },
  demoWrapper: { marginHorizontal: 16, marginBottom: 16, backgroundColor:
'#FFF', borderRadius: 8, padding: 12, elevation: 2 },
  demoJudul:   { fontSize: 13, fontWeight: '600', color: '#666', marginBottom:
8 },
  demoKonten:  { backgroundColor: '#EEF5FB', borderRadius: 4, padding: 8 },
  kotak:       { width: 60, height: 60, borderRadius: 6, justifyContent:
'center', alignItems: 'center', margin: 4 },
  kotakTeks:   { color: '#FFFFFF', fontWeight: 'bold', fontSize: 18 },
});
```

Catatan

Amati perbedaan tampilan setiap Demo di emulator. Perhatikan bagaimana justifyContent dan alignItems mengatur posisi kotak.

3 **Hubungkan ke App.js**

Import dan tampilkan FlexboxDemo di App.js:

```
import FlexboxDemo from './components/FlexboxDemo';

export default function App() {
  return <FlexboxDemo />;
}
```

4 **Eksperimen mandiri**

Tambahkan minimal 2 Demo baru ke FlexboxDemo.js dengan konfigurasi Flexbox yang berbeda dari yang sudah ada. Dokumentasikan hasilnya dengan screenshot.

Catatan

Coba: flexWrap: 'wrap' dengan banyak kotak, atau kombinasi alignSelf pada salah satu kotak.

Praktikum B — Membangun Layout Halaman Profil

Target & Waktu

Membuat halaman profil mobile lengkap menggunakan StyleSheet dan Flexbox.

Perkiraan waktu: 30 menit

9. Buat file components/HalamanProfil.js
10. Bangun layout halaman profil dengan struktur berikut:
    - Header berwarna biru gelap dengan nama dan NIM (warna putih)

- Section avatar: lingkaran berwarna berisi inisial nama (View dengan borderRadius setengah width)
- Section statistik: 3 kartu sejajar horizontal (Mata Kuliah, SKS, IPK) menggunakan flexDirection row
- Section info: daftar informasi (Program Studi, Semester, Email, No. HP) dengan layout label-nilai

11. Gunakan kode layout halaman dari materi 2.3 sebagai referensi — modifikasi sesuai kebutuhan
12. Pastikan menggunakan ScrollView agar konten bisa di-scroll
13. Terapkan shadow/elevation pada kartu statistik agar terlihat lebih nyata

Kriteria Layout yang Baik

Seluruh layar dimanfaatkan (flex: 1 pada container utama)
Tidak ada teks yang terpotong atau keluar layar
Padding dan margin konsisten (gunakan nilai yang sama, misal 16)
Warna kontras — teks gelap di background terang, atau sebaliknya
Kartu statistik sejajar rapi (flexDirection: row + justifyContent: space-between)

Praktikum C — Event Handling Interaktif

Target & Waktu

Membuat komponen interaktif yang menggabungkan berbagai jenis event handler.
Perkiraan waktu: 30 menit

14. Buat file components/FormLogin.js
15. Implementasikan form login dengan spesifikasi berikut:

```
// Spesifikasi komponen FormLogin:
// - State: email, password, isLoading, pesanError
// - TextInput email: keyboardType='email-address', autoCapitalize='none'
// - TextInput password: secureTextEntry={true}
// - Kedua input: border berubah warna saat onFocus (biru) dan onBlur (abu)
// - Tombol Login (Pressable): style berubah saat ditekan
// - Validasi saat tombol Login ditekan:
//   * Email harus mengandung '@' dan '.'
//   * Password minimal 6 karakter
//   * Jika tidak valid: tampilkan pesanError di bawah tombol
//   * Jika valid: Alert.alert('Berhasil', 'Login sukses!')
// - Tombol 'Lupa Password?' (onPress): Alert informatif
// - onLongPress pada tombol Login: Alert.alert('Info', 'Versi 1.0.0')
```

16. Tambahkan komponen Switch untuk opsi 'Ingat Saya':

```
import { Switch } from 'react-native';

const [ingatSaya, setIngatSaya] = useState(false);

// Penggunaan:
<View style={{ flexDirection: 'row', alignItems: 'center', gap: 8 }}>
```

```
    <Switch
      value={ingatSaya}
      onValueChange={(val) => setIngatSaya(val)}
      trackColor={{ false: '#CCCCCC', true: '#2E75B6' }}
      thumbColor={ingatSaya ? '#1F4E79' : '#FFFFFF'}
    />
    <Text>Ingat Saya</Text>
  </View>
```

17. Uji semua skenario berikut di emulator:
    - Login dengan email dan password valid
    - Login dengan email tanpa '@' — pastikan pesan error muncul
    - Login dengan password kurang dari 6 karakter
    - Tekan lama tombol Login — Alert versi muncul
    - Aktifkan Switch 'Ingat Saya' dan amati perubahan warna

Latihan Mandiri

Kerjakan minimal satu dari dua latihan berikut. Latihan 2 memberikan nilai lebih tinggi.

Latihan 1 — Aplikasi Kartu Nama Digital

Buat aplikasi kartu nama digital yang interaktif. Aplikasi terdiri dari dua bagian: form input di atas dan preview kartu nama di bawah yang berubah secara real-time.

Spesifikasi yang wajib dipenuhi:

18. Form input: Nama, Jabatan, Perusahaan, Email, No. HP — semua menggunakan TextInput
19. Preview kartu nama berubah real-time mengikuti input (controlled component dengan state)
20. Layout kartu nama: header warna, nama besar, jabatan, divider, info kontak di bawah
21. Tombol 'Bagikan' (onPress): Alert berisi semua data yang sudah diisi
22. Tombol 'Reset' (onLongPress): konfirmasi reset form dengan Alert dua tombol (Cancel / Reset)
23. Input nama mendapat onFocus: border berubah warna

Contoh Alert dengan Dua Tombol

```
Alert.alert(
  'Konfirmasi',
  'Yakin ingin mereset semua data?',
  [
    { text: 'Batal', style: 'cancel' },
    { text: 'Reset', onPress: () => handleReset(), style: 'destructive' },
  ]
);
```

Latihan 2 — Aplikasi Kuis Sederhana

Buat aplikasi kuis pilihan ganda yang berjalan di emulator Android. Aplikasi menampilkan pertanyaan satu per satu dan menghitung skor akhir.

Spesifikasi yang wajib dipenuhi:

24. Minimal 5 soal pilihan ganda bertema bebas, masing-masing 4 pilihan jawaban
25. Layout: nomor soal + progress bar teks (misal: 'Soal 2 dari 5') di atas
26. Pertanyaan di tengah, 4 tombol jawaban di bawah (layout grid 2x2 atau vertikal)
27. Saat jawaban dipilih (onPress): tampilkan warna hijau jika benar, merah jika salah
28. Tombol 'Lanjut' muncul setelah jawaban dipilih untuk ke soal berikutnya
29. Halaman hasil: tampilkan skor (X dari 5), pesan berdasarkan skor, dan tombol 'Ulangi Kuis'

Petunjuk Struktur Data Soal

```
const soalKuis = [
  {
    id: 1,
    pertanyaan: 'Apa kepanjangan dari JSX?',
    pilihan: ['JavaScript XML', 'Java Syntax Extension', 'JSON XML', 'JavaScript Extension'],
    jawabanBenar: 0,  // index pilihan yang benar
  },
  // ... soal lainnya
];
```

Tantangan Tambahan (Bonus)

Tambahkan timer countdown per soal (gunakan useState + useEffect untuk interval)
Simpan nama pemain menggunakan TextInput di halaman awal sebelum kuis
Animasikan pergantian soal (hint: gunakan state opacity atau translateX)

Lembar Kerja dan Dokumentasi

Pertanyaan Analitis

Jawab pertanyaan berikut berdasarkan pemahaman dan hasil praktikum Anda:

1. Mengapa React Native menggunakan flexDirection: 'column' sebagai default, sedangkan CSS web menggunakan 'row'? Menurut Anda, apa alasan desain tersebut?

2. Jelaskan perbedaan antara onChangeText dan onChange pada TextInput! Kapan Anda lebih memilih menggunakan masing-masing?

3. Apa keunggulan Pressable dibanding TouchableOpacity? Berikan contoh kasus nyata di mana Pressable lebih tepat digunakan!

4. Dalam praktikum C (FormLogin), mengapa validasi dilakukan di dalam fungsi handleLogin (saat tombol ditekan) dan bukan langsung di onChangeText? Apa pro dan kontra kedua pendekatan tersebut?

Dokumentasi Screenshot

Tempelkan screenshot hasil praktikum dari emulator Android di bawah ini:

Screenshot 1: Praktikum A — Tampilan semua Demo Flexbox (scroll untuk lihat semua variasi)

[Tempelkan screenshot emulator Android di sini]

Screenshot 2: Praktikum B — Halaman Profil lengkap (header, avatar, statistik, info)

[Tempelkan screenshot emulator Android di sini]

Screenshot 3: Praktikum C — FormLogin: tampilkan kondisi error dan kondisi Alert berhasil

[Tempelkan screenshot emulator Android di sini]

Screenshot 4: Latihan Mandiri — Hasil latihan yang dikerjakan

[Tempelkan screenshot emulator Android di sini]

Rubrik Penilaian

Komponen	Bobot	Kriteria
Praktikum A — Flexbox Demo	20%	7 demo tampil benar, ditambah 2 eksperimen mandiri
Praktikum B — Layout Profil	25%	Layout rapi: header, avatar, statistik 3 kolom, daftar info
Praktikum C — Form Login	25%	Validasi berfungsi, Pressable style berubah, Switch aktif
Latihan Mandiri	20%	Minimal 1 latihan selesai dan berfungsi di emulator
Pertanyaan Analitis	10%	Jawaban menunjukkan pemahaman, bukan copy-paste materi

Catatan Asisten / Dosen

Nilai Akhir Praktikum:

Tanda Tangan Asisten / Dosen